

Universidad Técnica Estatal de Quevedo (UTEQ)

Facultad de Ciencias de la Computación y Diseño Digital

Carrera de Ingeniería de Software

Asignatura: Aplicaciones Web — Período académico 2026–2027

Sistema de Gestión Bibliotecaria Web (SGB-SaaS)

Entrega Final — Informe Académico Completo

Java 21 / Spring Boot 4

Angular 17

PostgreSQL 16

Redis 7

OWASP

Integrantes:

Cajas Ibarra Irvin Marcelo	C.I.: 1251039606	ORCID: 0009-0001-7308-1185
Loor Medranda Marlon Taylor	C.I.: 0928087469	ORCID: 0009-0006-6093-224X
Panama Murillo Moises Antonio	C.I.: 1251334544	ORCID: 0009-0009-8113-242X

Docente-director:

Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.
gguerrero@uteq.edu.ec

Fecha de cierre:

17 de agosto de 2026

Repositorio:

<https://github.com/mloorm14/sgb-saas> — Rama: main

Tag de esta entrega: v1.0.0

Commit tagueado (completo):
<COMMIT-HASH>

DOI:

<DOI-SOFTWARE-V1.0.0>

SOFT-R-A — 5to Nivel — Proyecto Fin de Curso (PFC), Entrega Final

Índice general

Resumen	5
1. Introducción	6
1.1. Contexto y motivación	6
1.2. Preguntas de investigación	7
1.3. Objetivos	7
1.3.1. Objetivo general	7
1.3.2. Objetivos específicos	7
1.4. Contribuciones	8
1.5. Organización del documento	9
2. Diseño y arquitectura del sistema	10
2.1. Nivel 1 – Contexto del sistema	10
2.2. Nivel 2 – Contenedores	10
2.3. Nivel 3 – Componentes	11
2.4. Decisiones arquitectónicas documentadas (ADR)	12
2.5. Atributos de calidad (ISO/IEC 25010)	13
2.6. Matriz de trazabilidad – resumen por tipo de acceso a datos	14
3. Implementación	16
3.1. Procedimiento almacenado y su invocación desde Spring Data JPA	16
3.2. Manejo uniforme de errores (RFC 7807)	20
3.3. Configuración paramétrica del sistema	21
3.4. Estrategia de caché Redis del catálogo	22
3.5. Transporte del token de sesión en cookie <code>HttpOnly</code>	22
4. Ingeniería de requisitos	24
4.1. Stakeholders	24
4.2. Técnicas de elicitación	24
4.3. Los 43 requisitos: categorización MoSCoW	25
4.4. Proceso de validación: 100 % de los requisitos Must verificados	25
4.5. Estabilidad del conjunto de requisitos: <code>CHANGELOG-REQ.md</code>	26
4.6. Checklist de calidad de requisitos (INCOSE v4)	26
5. Amenazas a la validez	28
5.1. Validez de constructo	28
5.2. Validez interna	29
5.3. Validez externa	29

5.4. Validez de conclusión 30

Índice de figuras

Índice de cuadros

Resumen

PENDIENTE: se escribe al final, cuando el resto del documento esté completo. El resumen estructurado que exige el Bloque B.2 de la guía (problema, método, resultados principales con cifras reales, conclusión) tiene que sintetizar el documento completo – Introducción, Trabajos Relacionados, Diseño y Arquitectura, Implementación, Resultados, Ingeniería de Requisitos, Amenazas a la Validez, Discusión y Conclusiones – y varios de esos capítulos todavía no existen (ver la nota de estado en `docs/informe-final.tex`). Redactarlo ahora obligaría a inventar resultados y conclusiones que este documento todavía no sostiene, exactamente el tipo de contenido fabricado que este proyecto evita en el resto de su documentación.

Capítulo 1

Introducción

1.1 Contexto y motivación

La gestión de una biblioteca institucional o municipal – control de catálogo, préstamos, devoluciones, reservaciones y multas – se sigue resolviendo con frecuencia mediante procesos manuales o herramientas genéricas no pensadas para el dominio bibliotecario (hojas de cálculo, registros en papel, o sistemas de escritorio aislados sin acceso remoto). En el contexto concreto de una biblioteca universitaria como la de la Universidad Técnica Estatal de Quevedo (UTEQ), esto se traduce en fricción operativa real y verificable en el propio dominio del proyecto: un bibliotecario que registra un préstamo a mano no tiene forma automática de saber si un usuario ya tiene multas pendientes ni de decrementar el stock disponible de forma atómica; un lector no tiene forma de autoservicio para ver sus propios préstamos, reservar un libro o consultar su historial sin acudir físicamente al mostrador.

Nota de honestidad (estimación razonada, no dato de fuente externa verificada). Este documento evita deliberadamente citar una cifra de "población afectada" (número de usuarios de biblioteca, volumen de préstamos anuales, etc.) porque el equipo no cuenta con un estudio o fuente institucional verificada que la respalde – inventar una cifra cuantitativa aquí sería exactamente el tipo de dato fabricado que el resto de este proyecto se niega a producir (ver, por ejemplo, el criterio ya aplicado en `docs/checklists/incose2023-req.md` y en las notas de honestidad de `docs/requisitos/SRS-v1.0.0.md`). Lo que sí se sostiene con evidencia real es el perfil de carga: una biblioteca universitaria como la de UTEQ tiene un volumen de operación bajo y no sostenido en el tiempo (ráfagas puntuales en fechas concretas del calendario académico, no tráfico constante de alta concurrencia), supuesto ya declarado explícitamente en `docs/arquitectura/ISO25010.md` y usado para justificar decisiones de arquitectura (p. ej. la prioridad *Media*, no *Alta*, de la característica "Eficiencia de desempeño").

SGB-SaaS se construye como respuesta a ese problema concreto: una plataforma web (no de escritorio) que digitaliza el ciclo completo de préstamo/devolución/reserva/multa, con control de acceso basado en roles (RBAC) para los distintos perfiles reales de una biblioteca (lector, bibliotecario, gerente, administrador), y que – a diferencia de una implementación mínima que solo "funcione" – se desarro-

lla junto con un programa explícito de evaluación de calidad: pruebas de carga reales, auditoría de seguridad OWASP Top 10 en vivo, ingeniería de requisitos formal con trazabilidad verificable, y un protocolo de usabilidad (SUS) con participantes reales.

1.2 Preguntas de investigación

- **RQ1.** ¿En qué medida el uso de caché (Redis) reduce la latencia observable del endpoint de catálogo bajo carga concurrente controlada, y con qué margen se cumplen los umbrales de rendimiento exigidos?
- **RQ2.** ¿Qué proporción de los controles auditados del OWASP Top 10:2021 presenta hallazgos reales en la implementación, y en qué proporción de esos hallazgos el propio equipo aplicó una corrección verificable dentro del mismo ciclo de desarrollo?
- **RQ3.** ¿Cómo perciben usuarios reales (con roles representativos del dominio, p. ej. lector y bibliotecario) la usabilidad del sistema, medida con el instrumento estandarizado System Usability Scale (SUS)?
- **RQ4.** ¿Qué efecto tiene adoptar una estrategia híbrida de acceso a datos (CRUD elemental vía ORM, operaciones multi-tabla vía procedimientos almacenados) sobre la trazabilidad de requisitos y la cobertura de pruebas automatizadas, frente a un enfoque de solo ORM?

1.3 Objetivos

1.3.1 Objetivo general

Evaluar empíricamente la calidad del sistema SGB-SaaS – en sus dimensiones de rendimiento, seguridad, usabilidad y arquitectura de acceso a datos – mediante evidencia reproducible, versionada y verificable de forma independiente en el propio repositorio del proyecto.

1.3.2 Objetivos específicos

1. **OE1** (trazado a RQ1). Medir, mediante pruebas de carga controladas (k6, 50 VUs concurrentes, múltiples corridas independientes), la latencia p95 del endpoint de catálogo con y sin caché activo, y contrastar el resultado agregado contra los umbrales de aceptación exigidos.
2. **OE2** (trazado a RQ2). Auditar en vivo, contra el stack Docker real (no simulado), un subconjunto representativo de controles del OWASP Top 10:2021, documentando cada hallazgo con evidencia cruda y aplicando corrección verificable a los defectos reales detectados dentro del propio ciclo del proyecto.

3. **OE3** (trazado a RQ3). Aplicar el cuestionario System Usability Scale (SUS) a una muestra de participantes reales, bajo un protocolo de consentimiento informado ya definido, para obtener una puntuación de usabilidad percibida con validez estadística.
4. **OE4** (trazado a RQ4). Documentar y trazar, mediante Architecture Decision Records (ADR) y una matriz de trazabilidad verificada automáticamente en CI, la estrategia híbrida de acceso a datos, cuantificando qué proporción de los requisitos del sistema depende de cada mecanismo (ORM vs. procedimiento almacenado) y qué porcentaje cuenta con prueba automatizada real.
5. **OE5** (transversal a RQ1–RQ4). Construir y versionar toda la evidencia empírica del proyecto – scripts de medición, datos crudos, análisis estadístico – de forma reproducible, para que un tercero pueda regenerarla de forma independiente sin depender de capturas de pantalla ni de afirmaciones sin respaldo.

1.4 Contribuciones

Este trabajo entrega, con evidencia real ya versionada en el repositorio al momento de escribir este capítulo:

- Un sistema web funcional y completo (tag `v1.0.0`) con 43 requisitos especificados y trazados (`docs/trazabilidad/matriz.csv`: 28 funcionales, 15 no funcionales), de los cuales el 100 % de los requisitos de prioridad **Must** cuenta con evidencia de verificación real (ver [Capítulo 4](#)).
- Evidencia empírica reproducible y versionada como código, no como capturas de pantalla editadas a mano: scripts de prueba de carga (`k6/`), análisis estadístico con test pareado de Wilcoxon y tamaño de efecto de Cliff's delta (`scripts/perf-analysis.py`), cobertura de código real (JaCoCo) y auditoría de accesibilidad (Lighthouse).
- Un artefacto de software citable con identificador persistente: el software está archivado en Zenodo con DOI (`10.5281/zenodo.21712467` para la versión `v0.9.0-rc`; la versión `v1.0.0` de este documento se re-archivará al cierre de esta entrega, ver el placeholder de DOI en la portada).
- Un checklist de calidad de requisitos según el marco INCOSE v4 (características C1–C15) aplicado con honestidad metodológica completa: documenta explícitamente qué requisitos no cumplen cada característica y por qué, en vez de reportar cumplimiento universal (`docs/checklists/incose2023-req.md`).
- Documentación arquitectónica versionada como código fuente, no como imágenes estáticas: el modelo C4 completo (niveles 1–3) en Structurizr DSL (`docs/arquitectura/workspace.dsl`) y 13 Architecture Decision Records (`docs/adr/`).

1.5 Organización del documento

El resto de este documento se organiza como sigue. [Capítulo 2](#) describe el diseño arquitectónico del sistema mediante el modelo C4 en sus tres niveles, las decisiones arquitectónicas formalizadas como ADR, y el mapeo contra las características de calidad de producto de ISO/IEC 25010. [Capítulo 3](#) detalla decisiones críticas de implementación con evidencia de código real: la estrategia de configuración paramétrica, el manejo uniforme de errores según RFC 7807, la estrategia de caché y el transporte seguro del token de sesión. [Capítulo 4](#) resume el proceso completo de ingeniería de requisitos – stakeholders, técnicas de elicitación, los 43 requisitos categorizados, su validación empírica y su evaluación según el marco de calidad INCOSE v4. [Capítulo 5](#) analiza críticamente las amenazas a la validez de constructo, interna, externa y de conclusión de la evidencia empírica recolectada. Los capítulos de Trabajos Relacionados, Resultados, Discusión, Conclusiones y Declaraciones/Anexos se incorporarán en tareas de redacción posteriores, una vez completada la evidencia que dependen de ellos (ver la nota de estado en `docs/informe-final.tex`).

Capítulo 2

Diseño y arquitectura del sistema

El modelo arquitectónico de SGB-SaaS está versionado como código fuente en `docs/arquitectura/workspace.dsl` (Structurizr DSL, modelo C4), en sus tres niveles completos – Contexto, Contenedores y Componentes – que reemplazan los diagramas estáticos sin fuente versionada de la Entrega 1A (`docs/diagramas/diagrama_c4_capa1.png`, `diagrama_c4_capa2.jpeg`).

2.1 Nivel 1 – Contexto del sistema

Cinco actores interactúan con SGB-SaaS como caja negra: **Lector** (consulta el catálogo, reserva libros, ve su historial de préstamos y multas), **Bibliotecario** (registra préstamos, devoluciones y multas desde el mostrador; gestiona el catálogo), **Gerente** (supervisa la operación: cuentas de personal, catálogos maestros de estado y reportes de auditoría), **Administrador** (configura parámetros del sistema y catálogos base de la plataforma – rol que no existía en el diseño original de Entrega 1A, incorporado al normalizar el modelo RBAC) y **Visitante anónimo** (sin cuenta todavía; hoy solo puede registrarse o iniciar sesión).

El propio `workspace.dsl` documenta, en su comentario de cabecera, una discrepancia real entre el diseño original de Entrega 1A y la implementación actual que vale la pena señalar aquí: Gemini 2.0 Flash API y Google Books API aparecían como sistemas externos en el diseño original y se **retiraron** del diagrama actual porque no existe ninguna integración con Google Books en el código real (Gemini sí se integró, pero como parte interna del contenedor Backend, no como un "sistema externo" del Nivel 1 – ver Nivel 3 más abajo). Tampoco se diseñó nunca un endpoint de catálogo público accesible sin autenticación: `LibroController` exige rol `LECTOR` (o superior) incluso para listar libros, así que el alcance real del *Visitante anónimo* se limita a registro e inicio de sesión.

2.2 Nivel 2 – Contenedores

Cuatro contenedores, alineados 1:1 con `docker-compose.yml`:

- **Frontend Angular** (TypeScript/Angular 17) – SPA de catálogo, formularios reactivos y gestión de sesión con el `accessToken` en memoria, nunca en `localStorage`.
- **Backend Spring Boot** (Java 21/Spring Boot 4.0.6) – API REST con autenticación JWT (cookie `HttpOnly` para el `refreshToken`), autorización RBAC por roles, y lógica de negocio de préstamos/reservas/multas vía JPA y procedimientos SQL.
- **PostgreSQL 16** – usuarios, roles/permisos, catálogo, préstamos, reservas, multas y bitácora de auditoría (26 tablas).
- **Redis 7** – blacklist de tokens JWT revocados (logout / revocación) y caché del listado de libros con TTL configurable externamente.

Respecto al diseño original de Entrega 1A, el contenedor Redis y el actor Administrador son incorporaciones directas de la normalización RBAC y del mecanismo de revocación de tokens vía *blacklist*, ambos ausentes del diagrama de contenedores original.

2.3 Nivel 3 – Componentes

El Nivel 3 se completó una vez cerradas las 8 ramas del módulo de Préstamos/Reservas/Multas (evitando así reconstruir el diagrama dos veces mientras ese módulo seguía en desarrollo activo). Se construyó a partir del inventario real de *controllers/services/clases* de seguridad del backend – no del diseño aspiracional –, agrupado por dominio en 21 componentes para mantener el diagrama legible en vez de tener una entrada por cada una de las cerca de 30 clases reales:

- **Capa API** (7 componentes): Auth API, Catálogo API, Préstamos API, Notificaciones API, CredencialQR API, Chatbot API, Admin API – un componente por dominio funcional, no por clase `@RestController` individual.
- **Capa de servicio** (8 componentes): Auth Service, Catálogo Service, Préstamos Service, Reportes Service, Notificaciones Service, CredencialQR Service, Chatbot Service, Admin Service.
- **Seguridad** (5 componentes): `JwtService`, `JwtAuthFilter`, `UserDetailsServiceImpl`, `LoginRateLimiter` (OWASP A07, límite de intentos de login por correo+IP) y `ChatbotRateLimiter` (límite de mensajes al asistente por usuario).
- **Integración externa** (1 componente): `GeminiClient` – cliente HTTP hacia la API de Gemini 2.0 Flash, con reintento simple ante 429/timeout/5xx y sin exponer la URL con la API key en logs (ver ADR-016, [Sección 2.4](#)).

Las relaciones entre componentes se verificaron contra el código real (no se infirieron): cada flecha API → Service, Service → Postgres/ Redis, y las dependencias cruzadas de `ChatbotService` hacia `CatálogoService` y `PréstamosService` para el *grounding* real de sus respuestas (consulta disponibilidad de libros y reservas del propio usuario antes de responder, para no inventar disponibilidad) corresponden a llamadas reales entre las clases Java correspondientes.

Validación sintáctica y exportación visual

El archivo se validó sintácticamente con el motor activamente mantenido `structurizr/structurizr` (subcomando `validate`), no con `structurizr/cli` – que en este momento del ecosistema Structurizr es una imagen retirada que solo imprime un aviso de desuso y termina con código de salida 0 para cualquier entrada, válida o no, por lo que una validación histórica contra esa imagen no habría sido una validación real. El diagrama de Nivel 3 se exportó a SVG mediante el flujo `structurizr/structurizr export -format plantuml` seguido de renderizado con PlantUML, resultando en `docs/arquitectura/c4-nivel3-componentes.svg`, versionado junto al DSL fuente.

2.4 Decisiones arquitectónicas documentadas (ADR)

El repositorio contiene **13 Architecture Decision Records** (`docs/adr/`), agrupados a continuación según los 6 temas obligatorios del Bloque D de la guía (mapeo completo en `docs/adr/README.md`); dos de esos temas están cubiertos por un ADR cuya numeración interna del repositorio no coincide con la numeración sugerida por la guía (ADR-006/ADR-007 sugeridos por la guía para acceso a datos y despliegue corresponden a ADR-013 y ADR-012 en este repositorio), por continuidad con la numeración ya evaluada en la Tercera Entrega – ver la nota explícita en `docs/adr/README.md` sobre por qué no se renumeraron los archivos.

#	Tema obligatorio (Bloque D)	ADR(s)
1	Elección de la pila tecnológica principal	ADR-001
2	Esquema de autenticación	ADR-010 + ADR-003 + ADR-007
3	Gestor de base de datos	ADR-011
4	Estrategia de caché	ADR-008
5	Estrategia de despliegue	ADR-012
6	Estrategia de acceso a datos (CRUD-ORM vs. SP)	ADR-013

Los 7 ADR restantes no mapean a ninguno de los 6 temas obligatorios, pero documentan decisiones reales tomadas durante el desarrollo: ADR-006 (versionado de esquema reproducible, Flyway + snapshot), ADR-009 (licencia MIT), ADR-014 (separación de responsabilidades entre ADMIN y GERENTE: el primero administra permisos/configuración de plataforma, el segundo la operación diaria – ninguno de los dos queda excluido de *ver* el padrón de usuarios, pero solo ADMIN puede *modificarlo*), ADR-015 (TLS termina en el proxy, no en el backend Spring Boot: el backend queda preparado vía `server.forward-headers-strategy: framework` para reconocer X-Forwarded-Proto y activar HSTS el día que el proxy real active TLS, sin cambios de código adicionales), y ADR-016 (qué datos exactos viajan a la API externa de Gemini en el chatbot – texto del mensaje, historial de la sesión y un prompt de *grounding* con datos ya públicos del catálogo – y por qué eso no constituye una exposición indebida de información: nunca viajan credenciales, tokens, contraseñas ni identificación personal).

2.5 Atributos de calidad (ISO/IEC 25010)

`docs/arquitectura/ISO25010.md` prioriza las 8 características de calidad de producto de la norma para el contexto específico de este proyecto (biblioteca universitaria de bajo volumen, no un sistema de alta concurrencia), citando evidencia real donde existe en vez de estimarla. Se reproduce a continuación tal cual el documento fuente (única fuente de verdad de esta tabla; este capítulo no la duplica de memoria):

Característica	Prioridad	Estrategia / decisión que la atiende
Adecuación funcional	Alta	Procedimientos almacenados transaccionales para operaciones con lógica cruzada multi-tabla (<code>sp_registrar_devolucion</code> , <code>sp_pagar_multa</code> , <code>sp_anular_multa</code>), constraints de integridad referencial (FKs, CHECKs) en <code>db/schema.sql</code> .
Eficiencia de desempeño	Media	Caché Redis del listado de libros con TTL externo (ADR-008). Evidencia vigente: prueba de carga formal (50 VUs, 5 corridas de k6 con Wilcoxon + Cliff's delta) – p95 agregado 19,49 ms en <code>cache_caliente</code> (umbral <200 ms) y 7,50 ms en <code>cache_frio</code> (umbral <500 ms), 0 % de errores 5xx en 20 003 peticiones (<code>docs/mediciones/perf/REPORT.md</code>). El propio informe declara una limitación metodológica real (ver Capítulo 5): <code>cache_frio</code> mide sobre todo consultas con página vacía, no el costo real de traer una página llena sin caché. Una medición manual preliminar del 21 de julio había reportado cifras de otro orden de magnitud (~170 ms/~30 ms) por tratarse de una sola petición aislada sin carga concurrente; queda solo como antecedente histórico, no como evidencia vigente.
Compatibilidad	Media	API REST documentada con OpenAPI/Swagger vía <code>springdoc-openapi</code> (ADR-001), JSON como formato estándar – sin integración real hoy con sistemas académicos institucionales ni con Google Books API.
Usabilidad	Alta	<code>sp_registrar_devolucion</code> valida el estado del préstamo antes de actuar (rechaza con LB409 si ya estaba devuelto, evitando duplicidad); frontend Angular con formularios reactivos y validación antes de enviar.
Fiabilidad	Alta	Parcialmente atendida: ADR-003 documenta explícitamente que, si Redis cae, <code>JwtAuthFilter</code> queda sin forma de verificar revocaciones – la decisión <i>fail-open</i> vs. <i>fail-closed</i> sigue anotada como pendiente para producción, no resuelta. Los <i>healthchecks</i> de <code>docker-compose.yml</code> sí garantizan un orden de arranque correcto.

Característica	Prioridad	Estrategia / decisión que la atiende
Seguridad	Alta	Cookie <code>HttpOnly+Secure+SameSite=Strict</code> para el <code>refreshToken</code> (ADR-007), BCrypt costo 12, blacklist de tokens revocados (ADR-003), respuestas de error RFC 7807 sin fuga de detalles internos.
Mantenibilidad	Alta	Los ADR de <code>docs/adr/</code> documentan decisiones no obvias con sus alternativas consideradas; <code>docs/basedatos/CATALOGO-SP.md</code> documenta el criterio de separación CRUD-ORM vs. procedimientos. Nota de este informe¹ : la cifra real de ADRs es 13, no 6 – ver aclaración al pie.
Portabilidad	Alta	Docker Compose con imágenes base pinadas por digest sha256, <code>db/init/01-consolidado.sql</code> generado de forma reproducible, arranque de un solo comando vía <code>make up</code> .

2.6 Matriz de trazabilidad – resumen por tipo de acceso a datos

La matriz completa de los 43 requisitos vive en `docs/trazabilidad/matriz.csv` y se valida automáticamente en cada ejecución de CI vía `scripts/validate-traceability.sh`. Este capítulo no reproduce las 43 filas (el detalle completo por requisito vive en [Capítulo 4](#) y en la propia matriz); resume aquí la columna `tipo_acceso`, relevante para esta sección por ser la evidencia directa de que la estrategia híbrida CRUD-ORM/SP declarada en ADR-013 se aplica de verdad y no solo en teoría.

Nota de corrección respecto a una versión anterior de este capítulo. `matriz.csv` tenía 4 filas (REQ-F-018, REQ-F-019, REQ-F-020, REQ-F-028) con una coma sin escapar dentro de un campo, que rompía el parseo CSV estándar – defecto ya corregido en el commit `ecfaf52`. Una versión anterior de esta tabla, calculada antes de esa corrección, solo había detectado 2 de las 4 filas afectadas (REQ-F-019, REQ-F-020) y las marcaba como “sin clasificar”; además atribuía el problema de REQ-F-019 a la columna `tipo_acceso`, cuando en realidad ese campo sí tenía un valor de `tipo_acceso` válido (“CRUD-ORM + generacion de imagen (ZXing)”) – el campo roto en esa fila específica era `modulo_codigo`, no `tipo_acceso`. Con las 43 filas parseando limpio, la tabla de abajo reclasifica las 4 filas correctamente en vez de dejarlas fuera:

¹El texto original de `IS025010.md` en esta fila cita “6 ADRs existentes”, cifra que quedó desactualizada tras incorporar los 8 módulos construidos después de la Tercera Entrega – el repositorio contiene 13 ADRs a la fecha de este capítulo ([Sección 2.4](#)). Se transcribe el texto original de la fuente junto con esta aclaración en vez de editarlo silenciosamente, ya que corregir `IS025010.md` está fuera del alcance de esta tarea.

Tipo de acceso a datos	Requisitos	%
CRUD-ORM (exclusivo, incl. combinado con Redis/SMTP/caché/API externa)	21	48,8 %
Híbrido SP + CRUD-ORM (requisitos transversales)	2	4,7 %
Procedimiento/función SQL (SP, exclusivo, incl. combinado con generación de PDF)	8	18,6 %
No aplica (decisión de arquitectura, configuración, control de acceso o UI sin acceso directo a BD)	11	25,6 %
Redis + SMTP (sin tabla Postgres nueva; no aplica CRUD-ORM ni SP) ²	1	2,3 %
Total	43	100,0 %

De los 43 requisitos, 10 (23,3 %) involucran al menos un procedimiento o función SQL (8 exclusivamente, 2 en combinación con CRUD-ORM: **REQ-NF-004**, transversal a los módulos de préstamos/multas/reservaciones vs. auth/libros/bitácora, y **REQ-NF-013**, prevención de inyección SQL sobre ambos mecanismos), consistente con la justificación de ADR-013: reservar los procedimientos almacenados para operaciones con validación cruzada multi-tabla o transacciones atómicas compuestas (registrar préstamo, registrar devolución, pagar multa, anular multa, expirar reservaciones vencidas, y los 3 reportes agregados), dejando el CRUD elemental de una sola tabla en Spring Data JPA. **REQ-F-018** (renovación de préstamo), pese a mencionar textualmente “SP” en su valor de `tipo_acceso` (“CRUD-ORM (validación en Java, sin SP nuevo)”), es CRUD-ORM exclusivo: la frase aclara explícitamente que la renovación *no* introdujo un procedimiento almacenado nuevo, la validación de sus 3 reglas de negocio corre en Java – se señala aquí porque una primera pasada de clasificación automática por coincidencia de texto la contó por error como híbrida, hasta revisar el valor completo del campo.

²**REQ-F-020**: `tipo_acceso` = “Redis (código OTP con TTL, sin tabla nueva) + SMTP (EmailService)”. Categoría propia, no un defecto de la matriz – el código de verificación vive en Redis con TTL, sin persistirse en una tabla Postgres nueva, y el envío usa SMTP directo; no encaja limpiamente en CRUD-ORM ni en SP.

Capítulo 3

Implementación

Este capítulo documenta decisiones críticas de implementación del backend, cada una con al menos un listado de código real extraído directamente del repositorio.

3.1 Procedimiento almacenado y su invocación desde Spring Data JPA

La estrategia híbrida de acceso a datos (ADR-013, [Sección 2.4](#)) exige que las operaciones con validación cruzada multi-tabla se implementen como procedimiento o función SQL. El siguiente listado es `db/procs/sp_crear_prestamo.sql` completo: registra un préstamo de forma atómica, validando que el usuario no esté bloqueado por multas y que el libro tenga stock disponible, usando SQLSTATE personalizados (LB404/LB422) para que el backend traduzca el error sin parsear texto.

Listing 3.1: `db/procs/sp_crear_prestamo.sql` (completo)

```
1 CREATE OR REPLACE FUNCTION sp_crear_prestamo(  
2     p_usuario_id BIGINT,  
3     p_libro_id BIGINT,  
4     p_bibliotecario_id BIGINT,  
5     p_dias_prestamo INTEGER  
6 )  
7 RETURNS BIGINT  
8 LANGUAGE plpgsql  
9 AS $$  
10 DECLARE  
11     v_estado_usuario_id      INTEGER;  
12     v_estado_bloqueado_id    INTEGER;  
13     v_stock_disponible       SMALLINT;  
14     v_estado_activo_prestamo_id INTEGER;  
15     v_prestamo_id            BIGINT;  
16 BEGIN  
17     SELECT id INTO v_estado_bloqueado_id  
18         FROM estados_usuario  
19         WHERE nombre = 'BLOQUEADO_POR_MULTA';
```

```
20
21     SELECT estado_id INTO v_estado_usuario_id
22     FROM usuarios
23     WHERE id = p_usuario_id
24     FOR UPDATE;
25
26 IF NOT FOUND THEN
27     RAISE EXCEPTION 'El usuario % no existe', p_usuario_id
28     USING ERRCODE = 'LB404';
29 END IF;
30
31 IF v_estado_usuario_id = v_estado_bloqueado_id THEN
32     RAISE EXCEPTION 'El usuario % esta bloqueado por multas
33     pendientes ...',
34     p_usuario_id USING ERRCODE = 'LB422';
35 END IF;
36
37 SELECT stock_disponible INTO v_stock_disponible
38 FROM libros WHERE id = p_libro_id FOR UPDATE;
39
40 IF NOT FOUND THEN
41     RAISE EXCEPTION 'El libro % no existe', p_libro_id
42     USING ERRCODE = 'LB404';
43 END IF;
44
45 IF v_stock_disponible <= 0 THEN
46     RAISE EXCEPTION 'El libro % no tiene stock disponible',
47     p_libro_id
48     USING ERRCODE = 'LB422';
49 END IF;
50
51 UPDATE libros SET stock_disponible = stock_disponible - 1
52 WHERE id = p_libro_id;
53
54 INSERT INTO prestamos (usuario_id, libro_id,
55     bibliotecario_id,
56     fecha_prestamo, fecha_devolucion_estimada,
57     estado_prestamo_id)
58 VALUES (p_usuario_id, p_libro_id, p_bibliotecario_id, NOW(),
59     NOW() + make_interval(days => p_dias_prestamo),
60     v_estado_activo_prestamo_id)
61 RETURNING id INTO v_prestamo_id;
62
63 RETURN v_prestamo_id;
64 END;
65 $$;
```

Nota de honestidad sobre el mecanismo de invocación. La guía de esta entrega espera que el método Java invoque el procedimiento con la anotación `@Procedure` de Jakarta Persistence. En este repositorio, ese *fue* el mecanismo original, pero se reemplazó por `@Query(nativeQuery = true)` tras un fallo real en tiempo

de ejecución: Hibernate genera sintaxis de parámetros nombrados de PostgreSQL (nombre => valor) dentro del escape JDBC {call ...} usado por @Procedure, que el driver pgjdbc no soporta ahí – PrestamoMultaProcedureIntegrationTest (test de integración real contra PostgreSQL, no mocks) confirmó el fallo en la primera ejecución. La migración de los 5 métodos afectados está documentada en docs/mediciones/backend/2026-07-28-fallo-invocacion-sp-multi-out.md y referenciada como *issue* conocido de Spring Data JPA (spring-projects/spring-data-jpa#3393). El siguiente listado muestra el método real vigente hoy en PrestamoProcedureRepository.java, incluyendo el comentario del propio código que documenta por qué ya no usa @Procedure:

Listing 3.2: PrestamoProcedureRepository.spCrearPrestamo (mecanismo vigente)

```

1  /**
2   * sp_crear_prestamo: retorno escalar unico (BIGINT). Antes
3   * usaba
4   * @Procedure(procedureName=...) -- fallaba en runtime con
5   * "syntax error at or near '=>'" porque Hibernate genera
6   * sintaxis de
7   * parametros nombrados de PostgreSQL dentro del escape JDBC
8   * {call ...}, que pgjdbc no soporta. @Query nativa con SELECT
9   * evita
10  * el CallableStatement por completo.
11  */
12  @Query(value = "SELECT sp_crear_prestamo(:p_usuario_id, :
13      p_libro_id, " +
14          ":p_bibliotecario_id, :p_dias_prestamo)",
15      nativeQuery = true)
16  Long spCrearPrestamo(
17      @Param("p_usuario_id") Long usuarioId,
18      @Param("p_libro_id") Long libroId,
19      @Param("p_bibliotecario_id") Long bibliotecarioId,
20      @Param("p_dias_prestamo") Integer diasPrestamo
21  );

```

Los parámetros siguen bindeándose por nombre (:p_usuario_id, etc.) exactamente igual que con @Procedure – la única diferencia es el mecanismo JDBC que usa Hibernate para transportar la llamada (PreparedStatement en vez de CallableStatement), por lo que la prohibición de SQL dinámico o concatenación de entrada de usuario (regla A.2.3 de la guía) se sigue cumpliendo igual (docs/basedatos/CATALOGO-SP.md, sección “Nota de diseño: por qué 5 usan @Procedure...y 2 usan @Query”). De los 7 objetos SQL del catálogo, los 5 con efectos secundarios usaban originalmente @Procedure/@NamedStoredProcedureQuery y hoy usan @Query(nativeQuery = true) tras esta migración; las 2 funciones de solo lectura que retornan TABLE (varias filas) usaron @Query nativa desde el principio, porque la API de *stored procedures* de JPA 2.1 no tiene una forma estándar de exponer un RETURNS TABLE de PostgreSQL a través de @Procedure.

¿Esto viola la prohibición de SQL dinámico de la guía (A.2.1)?

No. La guía (Bloque A.2.1) establece textualmente que “queda prohibido invocarlos mediante concatenación de cadenas en `createNativeQuery(...)`”. Esa prohibición apunta a un patrón concreto y distinto del usado en este proyecto: construir la sentencia SQL en tiempo de ejecución **concatenando texto de entrada del usuario** dentro del propio literal de la consulta – p. ej. `createNativeQuery("SELECT sp_crear_prestamo(" + idUsuario + ")")`, donde `idUsuario` pasa a formar parte literal de la cadena SQL antes de ejecutarse, el vector clásico de inyección SQL.

El código real de este proyecto **no hace eso en ningún punto**. El listado 3.2 usa `@Query(value = "SELECT sp_crear_prestamo(:p_usuario_id, ...)", nativeQuery = true)` con **parámetros nombrados bindeados** (`:p_usuario_id`, `:p_libro_id`, ...) resueltos por Spring Data JPA/Hibernate vía `PreparedStatement` antes de llegar al driver JDBC – el valor de cada parámetro nunca se concatena, formatea ni interpola dentro del literal de la cadena SQL; el literal que Java compila es fijo (`"SELECT sp_crear_prestamo(:p_usuario_id, :p_libro_id, :p_bibliotecario_id, :p_dias_prestamo)"`) y el binding ocurre por separado, exactamente como con `@Procedure`. Esta forma de `@Query` nativo con parámetros nombrados es, en sí misma, un mecanismo formal y documentado de Spring Data JPA para invocar funciones/procedimientos parametrizados – no es un atajo informal ni una forma disimulada de SQL dinámico.

En síntesis: **la regla A.2.1 prohíbe construir SQL concatenando entrada de usuario dentro del texto de la consulta; los parámetros bindeados de `@Query` nativa no construyen SQL a partir de input en absoluto, por lo que este patrón no cae bajo esa prohibición**. El motivo del cambio desde `@Procedure`/`@NamedStoredProcedureQuery` tampoco fue estilístico: está documentado con evidencia real y verificable en `docs/mediciones/backend/2026-07-28-fallo-invocacion-sp-multi-out.md` (fallo reproducido en runtime, con el mensaje de error real de PostgreSQL/pgjdbc) y corresponde a un defecto conocido de la combinación Hibernate 6.2+/7.x con procedimientos multi-OUT en PostgreSQL, reportado en el repositorio público de Spring Data JPA (`spring-projects/spring-data-jpa#3393`) – no una decisión arbitraria del equipo para eludir el mecanismo que pide la guía.

Nota sobre verificación automática. Al momento de escribir este capítulo no existe todavía en el repositorio un script de análisis estático tipo `scripts/audit-sql-dynamic.sh` que detecte patrones de SQL dinámico en el código fuente – se deja constancia aquí para que, cuando se incorpore, su regla de detección distinga explícitamente entre concatenación real de entrada de usuario dentro del texto de una consulta (el patrón que sí debe marcar) y `@Query(nativeQuery = true)` con parámetros nombrados bindeados (`:p_...`) como el de esta sección, que no debería marcarse como hallazgo – un análisis ingenuo basado solo en buscar la cadena `nativeQuery = true` sin inspeccionar si el valor de `value` contiene variables Java concatenadas produciría un falso positivo contra este patrón, ya verificado como seguro.

3.2 Manejo uniforme de errores (RFC 7807)

Todas las respuestas de error del backend se devuelven como `ProblemDetail` (RFC 7807), vía un único `@RestControllerAdvice` (`GlobalExceptionHandler`). Un caso particularmente representativo es la traducción de los `SQLSTATE` personalizados de los procedimientos almacenados (Sección 3.1) a códigos HTTP, sin que el backend tenga que parsear el texto del mensaje de error:

Listing 3.3: `GlobalExceptionHandler.handleStoredProcedureError` (extracto real)

```
1 @ExceptionHandler({
2     InvalidDataAccessResourceUsageException.class,
3     UncategorizedSQLException.class,
4     DataAccessException.class
5 })
6 public ProblemDetail handleStoredProcedureError(Exception ex) {
7     Throwable causa = ex;
8     while (causa != null && !(causa instanceof SQLException)) {
9         causa = causa.getCause();
10    }
11
12    if (causa instanceof SQLException sqlEx) {
13        String sqlState = sqlEx.getSQLState();
14        HttpStatus status = switch (sqlState) {
15            case "LB404" -> HttpStatus.NOT_FOUND;
16            case "LB409" -> HttpStatus.CONFLICT;
17            case "LB422" -> HttpStatus.UNPROCESSABLE_ENTITY;
18            default -> null;
19        };
20        if (status != null) {
21            return ProblemDetail.forStatusAndDetail(status,
22                sqlEx.getMessage());
23        }
24
25        log.error("Error no controlado en procedimiento almacenado",
26            ex);
27        return ProblemDetail.forStatusAndDetail(
28            HttpStatus.INTERNAL_SERVER_ERROR, "Error interno del
29            servidor");
30    }
31 }
```

Sin este manejador dedicado, cualquier `RAISE EXCEPTION ... USING ERRCODE = 'LBxxx'` de un procedimiento llegaba envuelto en una `DataAccessException` genérica de Spring, y caía en el *catch-all* de último recurso – un error de negocio real (p. ej. “préstamo ya devuelto”) se veía como un 500 genérico en vez de un 409 con el mensaje real. El mismo patrón de un `@ExceptionHandler` dedicado por tipo de excepción se repite para excepciones de dominio Java (p. ej. `CorreoYaRegistradoException` → 409, `LoginRateLimitExcedidoException` → 429) y para excepciones del propio framework que antes caían también en el *catch-all*: `AuthorizationDeniedException` (autorización de método vía `@PreAuthorize`,

antes producía 500 en vez de 403) y `NoResourceFoundException` (recurso estático inexistente bajo el perfil `prod`, corregido en esta misma entrega – ver `docs/mediciones/sec/2026-08-11-owasp-a05-verificacion-real.md`).

3.3 Configuración paramétrica del sistema

`ConfiguracionSistemaService` expone parámetros de la tabla `configuracion_sistema` (p. ej. el máximo de renovaciones de un préstamo, ver `REQ-F-018`) para leerse en cada operación de negocio sin ir a la base de datos cada vez, con un caché en memoria simple (`ConcurrentHashMap` por instancia, no Redis) invalidado clave por clave al escribir:

Listing 3.4: `ConfiguracionSistemaService` (extracto real: lectura cacheada y escritura)

```
1 private final ConcurrentHashMap<String, String> cache = new
    ConcurrentHashMap<>();
2
3 @Transactional(readOnly = true)
4 public String obtenerValor(String clave) {
5     String cacheado = cache.get(clave);
6     if (cacheado != null) {
7         return cacheado;
8     }
9     String valor = repo.findById(clave)
10         .orElseThrow(() -> new EntityNotFoundException(
11             CLAVE_NO_ENCONTRADA + clave))
12         .getValor();
13     cache.put(clave, valor);
14     return valor;
15 }
16
17 @Transactional
18 public ConfiguracionSistemaResponseDTO actualizar(String clave,
19     String nuevoValor) {
20     ConfiguracionSistema config = repo.findById(clave)
21         .orElseThrow(() -> new EntityNotFoundException(
22             CLAVE_NO_ENCONTRADA + clave));
23     config.setValor(nuevoValor);
24     repo.save(config);
25     cache.remove(clave);
26     return new ConfiguracionSistemaResponseDTO(config.getClave(),
27         config.getValor());
28 }
```

La elección deliberada de un `ConcurrentHashMap` local en vez de Redis (que sí se usa para el caché del catálogo, ver [Sección 3.4](#)) está documentada en el propio Javadoc de la clase: este valor cambia con muy poca frecuencia (un ADMIN ajustando un parámetro puntual), a diferencia del caché “libros”, que sí necesita compartirse entre instancias e

invalidarse por TTL. Además, actualizar solo permite modificar claves **ya existentes** – el catálogo de parámetros válidos lo define el dominio (migraciones/*seed*), no quien llama al endpoint, evitando que quede una clave suelta sin ningún consumidor real.

3.4 Estrategia de caché Redis del catálogo

El listado de libros usa caché declarativo de Spring (@Cacheable/@CacheEvict) sobre Redis, con TTL configurable externamente (ADR-008):

Listing 3.5: LibroService (extracto real: lectura cacheada y evicción en escritura)

```
1 @Cacheable("libros")
2 @Transactional(readOnly = true)
3 public Page<LibroResponseDTO> listar(Pageable pageable) {
4     return libroRepo.findByEstado_Nombre(ESTADO_ACTIVO, pageable
5         )
6         .map(this::toDTO);
7 }
8 @CacheEvict(value = "libros", allEntries = true)
9 @Transactional
10 public LibroResponseDTO crear(LibroRequestDTO dto) {
11     if (libroRepo.existsByIsbn(dto.isbn())) {
12         throw new IllegalArgumentException("ISBN ya registrado:
13             " + dto.isbn());
14     }
15     validarStock(dto.stockTotal(), dto.stockDisponible());
16     return toDTO(libroRepo.save(fromDTO(dto)));
17 }
```

crear, actualizar y eliminar invalidan el caché completo del listado (`allEntries = true`) para no dejar una respuesta paginada obsoleta tras cualquier escritura. El método `listarPorCategoria` (filtro por categoría/autor) deliberadamente **no** lleva @Cacheable: combinar el filtro con el caché exigiría una clave compuesta fuera del alcance de la rama que lo introdujo, y no es el camino más transitado del catálogo – decisión documentada como comentario en el propio código, no un descuido. El método `sugerir` (autocompletado) usa un caché *distinto* ("sugerencias-libros") con TTL propio, mucho más corto (5–10 s), porque su patrón de invalidación (una entrada por texto de búsqueda) es incompatible con el caché de listado general.

3.5 Transporte del token de sesión en cookie HttpOnly

El `refreshToken` viaja exclusivamente en una cookie `HttpOnly`, `Secure`, `SameSite=Strict`, con `path=/api/auth` (ADR-007) – nunca en el cuerpo JSON, para que JavaScript del frontend no pueda leerlo ni reenviarlo manualmente:

Listing 3.6: AuthController.buildRefreshCookie (real, completo)


```
1 private ResponseCookie buildRefreshCookie(String valor, long
   maxAgeMs) {
2     return ResponseCookie.from(REFRESH_COOKIE_NAME, valor)
3         .httpOnly(true)
4         .secure(true)
5         .sameSite("Strict")
6         .path("/api/auth")
7         .maxAge(Duration.ofMillis(maxAgeMs))
8         .build();
9 }
```

El mismo método se reutiliza para *limpiar* la cookie en `logout` (`buildRefreshCookie("", 0)`, valor vacío y `maxAge=0`). El `accessToken`, de vida corta (1h), sigue viajando en el cuerpo JSON de la respuesta – migrarlo también a cookie está deliberadamente diferido (ver la nota de honestidad de ADR-007 y REQ-NF-002, [Capítulo 4](#)) por el impacto que tendría en `jwt.interceptor.ts/auth.service.ts` del frontend, no por un olvido.

Capítulo 4

Ingeniería de requisitos

4.1 Stakeholders

El proyecto identifica cinco grupos de interesados reales, no hipotéticos: los cuatro roles de usuario final del sistema – **Lector** (estudiante o miembro de la comunidad universitaria), **Bibliotecario** (personal de mostrador, bajo conocimiento técnico esperado), **Gerente** (responsable de la biblioteca, además de las funciones de Bibliotecario puede anular multas y ver reportes) y **Administrador** (administrador técnico/institucional del sistema) – más el **docente-director** (Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.) como evaluador académico del PFC y fuente directa de retroalimentación formal (ver las observaciones de Entrega 1A, Entrega 1B y Tercera Entrega en `docs/observaciones/OBSERVACIONES.md`). El propio equipo de desarrollo (3 integrantes) actúa simultáneamente como responsable de elicitación, implementación y verificación – una restricción real de recursos declarada explícitamente en `docs/requisitos/SRS-v1.0.0.md` (sección 2.4) que explica, entre otras cosas, por qué ciertos requisitos quedan con evidencia parcial en vez de simularse como completos.

4.2 Técnicas de elicitación

El proyecto combina tres fuentes de requisitos, ninguna ficticia:

- **Historias de usuario y casos de uso**, en formato Connextra (“Como *rol*, quiero *acción*, para *beneficio*”) y Cockburn respectivamente, versionados como un archivo por HU/CU en `docs/requisitos/historias/` y `docs/requisitos/casos-de-uso/` para el módulo original del equipo, y consolidados en dos archivos únicos (`historias-usuario.md`, `casos-de-uso.md`) para las 5 HU/CU del módulo de Préstamos/Reservas/Multas – una inconsistencia real de convención entre módulos, documentada explícitamente en el SRS en vez de normalizada en silencio.
- **Hallazgos de auditoría de seguridad como fuente directa de requisitos no funcionales**: REQ-NF-006 (*rate limiting* de login) y REQ-NF-007 (auditoría de eventos de autenticación) no eran requisitos preexistentes – nacieron de un

hallazgo real de la auditoría OWASP A07/A09 (ausencia total de límite de intentos y de registro de eventos), corregido dentro del mismo ciclo de desarrollo.

- **Inspección directa del código como último recurso declarado**, únicamente para los módulos construidos después de la Tercera Entrega que no llegaron a tener HU/CU dedicada – nunca presentada como si una HU real existiera. De los 43 requisitos, 8 (REQ-F-010, REQ-F-020 a REQ-F-022, REQ-F-025 a REQ-F-028) no citan ninguna HU/CU en la matriz de trazabilidad, y otros 5 (REQ-F-017, REQ-F-018, REQ-F-019 parcialmente, REQ-F-023, REQ-F-024) citan un identificador de HU/CU que **no corresponde a ningún archivo real** del repositorio, verificado por búsqueda exhaustiva antes de escribir el SRS – en ambos casos, el rationale de esos requisitos en el SRS se declara explícitamente como inferido de la implementación, no como un hecho documentado previamente.

4.3 Los 43 requisitos: categorización MoSCoW

La fuente única de verdad es `docs/trazabilidad/matriz.csv`, validada automáticamente en cada ejecución de CI por `scripts/validate-traceability.sh`. Este capítulo resume su distribución agregada; el detalle de cada requisito individual (descripción, rationale, criterio de aceptación medible, método de verificación) vive en `docs/requisitos/SRS-v1.0.0.md` y en la propia matriz – no se reproducen aquí las 43 fichas completas.

Dimensión	Categoría	Cantidad (%)
Tipo	Funcional (REQ-F)	28 (65,1 %)
	No funcional (REQ-NF)	15 (34,9 %)
Prioridad MoSCoW	Must	23 (53,5 %)
	Should	20 (46,5 %)
	Could	0 (0,0 %)
	Won't	0 (0,0 %)
Estado (matriz)	verificado	23 (53,5 %)
	implementado	20 (46,5 %)
Total		43 (100,0 %)

Ningún requisito usa las categorías **Could** o **Won't** de MoSCoW – dato real, no una omisión de esta tabla: los 43 requisitos del sistema se consideraron, al momento de incorporarse, o bien imprescindibles (**Must**) o bien deseables dentro del alcance actual (**Should**); ninguno se registró formalmente como descartado o como candidato de prioridad baja.

4.4 Proceso de validación: 100 % de los requisitos Must verificados

Al cierre de este capítulo, los 23 requisitos de prioridad **Must** tienen estado **verificado** en la matriz – es decir, cuentan con prueba automatizada real y/o evidencia empírica directa contra el stack en ejecución, no solo con código que compila. Este 100 % es

reciente: los dos últimos requisitos **Must** en estado **implementado** (REQ-F-001, registro de usuario, y REQ-NF-013, prevención de inyección SQL) carecían de prueba automatizada de regresión para parte de su criterio de aceptación – REQ-F-001 solo tenía cubierto el criterio de rechazo por correo duplicado, no el flujo exitoso ni el rechazo por contraseña corta; REQ-NF-013 solo tenía verificación manual puntual de la auditoría original, sin test de regresión permanente. Se cerraron agregando 2 tests nuevos (`AuthControllerTest.registro_passwordCorta_devuelve400ProblemDetail` y `AuthServiceTest.registroConPayloadDeInyeccionSql_seGuardaComoTextoLiteralSinLanzarExcepcion`, este último una regresión permanente del payload de inyección SQL verificado manualmente en la auditoría OWASP A03 original) en el commit `e1f0c25`.

4.5 Estabilidad del conjunto de requisitos: CHANGELOG-REQ.md

`docs/requisitos/CHANGELOG-REQ.md` compara el conjunto de requisitos entre la Tercera Entrega (`docs/requisitos/historico/SRS-v0.9.0-rc.md`, 30 requisitos) y esta entrega (`SRS-v1.0.0.md`, 43 requisitos), comparando el **cuerpo completo** de cada requisito compartido, no solo su título – metodología que descartó explícitamente 2 falsos positivos (REQ-F-016, REQ-NF-009) que una primera versión del script de comparación marcó como “modificados” por un artefacto de extracción de texto (un separador Markdown capturado como parte del cuerpo), documentado en el propio `CHANGELOG-REQ.md` en vez de contar una modificación que en realidad no ocurrió. El resultado: 13 requisitos agregados, 2 genuinamente modificados (REQ-NF-012/TLS y REQ-NF-014/CSP, ambos reclasificados de “pendiente” a “parcialmente implementado” tras cerrar parte de su alcance), 0 eliminados – **tasa de estabilidad** = $1 - (2/43) = 0,9535 \approx 0,953$, es decir, **95,3 %**.

Nota de honestidad (hallazgo de este capítulo, no de CHANGELOG-REQ.md). Ese mismo archivo reporta también un “porcentaje verificado” de 21/43, es decir 48,8 %, calculado en el momento en que se escribió. Esa cifra quedó desactualizada por el propio cierre descrito en la sección anterior: tras el commit `e1f0c25` (posterior a `CHANGELOG-REQ.md`), el conteo real de requisitos **verificado** es 23/43, es decir 53,5 %, no 48,8 %. Se señala aquí en vez de propagar silenciosamente la cifra desactualizada de la fuente citada – `CHANGELOG-REQ.md` en sí no se corrige en esta tarea, por estar fuera de su alcance.

4.6 Checklist de calidad de requisitos (INCOSE v4)

`docs/checklists/incose2023-req.md` evalúa los 43 requisitos contra las 9 características individuales (C1–C9) del marco INCOSE v4 y las 6 características de conjunto (C10–C15), con el mismo criterio de honestidad metodológica del resto del proyecto: no se marca cumplimiento por defecto.

Resultado agregado, considerando solo las 8 características de contenido

(C1–C8): **20 de 43 requisitos (46,5 %)** pasan las 8 sin ninguna excepción; los 23 restantes (53,5 %) tienen al menos una excepción documentada con evidencia concreta – típicamente C5 (*Singular*: un requisito agrupa más de una acción o regla de negocio distinguible bajo un mismo id, 12 casos) o C2 (*Appropriate*: criterio de aceptación demasiado escueto frente a requisitos hermanos del mismo módulo, 5 casos); las de C8 (*Correct*) son las menos frecuentes (4 casos) pero las más específicas, incluyendo una que este mismo checklist encontró antes de que se cerrara: la afirmación de REQ-F-001/REQ-NF-013 de que ciertos criterios carecían de prueba automatizada, ya resuelta en el commit `e1f0c25` citado en la sección anterior.

Hallazgo sistémico de C9 (*Conforming*), declarado sin ambigüedad: ninguno de los 43 requisitos está redactado como una única cláusula “[condición] [sujeto] *shall* [acción] [objeto] [restricción]” del patrón formal de ISO/IEC/IEEE 29148:2018. El formato real del SRS separa *Descripción* (sujeto + modal + acción + objeto, en prosa) de *Criterio de aceptación medible* (las condiciones/restricciones, en bullets aparte) – un híbrido Volere/Cockburn/Gherkin, no el patrón EARS de una sola oración. El propio checklist documenta esto como una desviación **estructural y sistemática**, no un defecto puntual de contenido de alguno de los 43 – razón por la cual, si se cuentan las 9 características en vez de solo las 8 de contenido, el resultado formal es 0 de 43 requisitos sin ninguna excepción (100 % con al menos una), enteramente explicado por este hallazgo de formato, no por 43 fallas de contenido independientes. No ocultar este resultado – aunque a primera vista parezca alarmante – es exactamente el criterio de honestidad metodológica que el propio checklist declara como su objetivo.

Sobre el conjunto (C10–C15): 4 de las 6 características de conjunto se evalúan como cumplidas con matices (*Complete, Feasible, Comprehensible, Able to be validated*), respaldadas por evidencia directa (los 43 requisitos ya implementados y validados en CI vía `scripts/validate-traceability.sh`). Las otras 2 (*Consistent, Correct*) se marcan explícitamente con excepción: se identificaron 2 inconsistencias reales dentro del conjunto – la contradicción ya autodeclarada entre REQ-F-001/REQ-F-020 sobre el estado inicial de una cuenta tras el registro, y una nueva (encontrada por el propio checklist, no señalada antes) entre las dos cifras de latencia citadas por REQ-NF-003 para la misma condición de caché frío.

Capítulo 5

Amenazas a la validez

Este capítulo analiza, en las 4 categorías estándar de amenazas a la validez de un estudio empírico de ingeniería de software, las limitaciones **reales** ya documentadas en el repositorio – no una lista genérica de amenazas de relleno que no correspondan a algo efectivamente observado o declarado durante este proyecto.

5.1 Validez de constructo

La amenaza de constructo más concreta de este proyecto está en el propio diseño del experimento de rendimiento (Sección 3.4, docs/mediciones/perf/REPORT.md): el escenario `cache_frio` se diseñó para medir el costo de una consulta al catálogo *sin* caché (el constructo de interés, para contrastarlo contra `cache_caliente`), pero su implementación real – un PRNG que elige una página aleatoria entre 0 y 999 en cada petición – hace que la gran mayoría de esas páginas queden fuera de rango de la tabla `libro` (poblada con datos de ejemplo de desarrollo, no con miles de registros), de modo que Spring Data devuelve una `Page` vacía sin necesidad de traer filas reales. En la práctica, `cache_frio` mide sobre todo el costo de una consulta `COUNT + SELECT` con resultado vacío, **no** el costo real de traer y serializar una página completa de libros sin caché – una construcción operacional distinta de la que el nombre del escenario sugiere. La comparación “caché caliente vs. caché frío” de este experimento, por tanto, **no aísla limpiamente el efecto del caché**: parte de la diferencia de latencia observada entre ambos escenarios refleja también el costo distinto de las consultas que cada uno ejecuta, no únicamente la presencia o ausencia de caché. Esta limitación está documentada explícitamente en el propio `REPORT.md` (punto 3 de su “Análisis breve”), no descubierta recién en este capítulo – se reproduce aquí porque es exactamente el tipo de amenaza de constructo que este capítulo debe declarar, en vez de reemplazarla por una amenaza genérica que no corresponda al proyecto real.

5.2 Validez interna

La corrida 1 de las 5 corridas de k6 ([Capítulo 2](#), [docs/mediciones/perf/REPORT.md](#)) presenta una cola de latencia anómala en `cache_caliente` (p95=107,31 ms, frente a un rango de 6,25–12,36 ms en las 4 corridas siguientes) que no se repite en ninguna corrida posterior. La explicación documentada es *warm-up* de JVM/JIT y del *pool* de conexiones de HikariCP: la corrida 1 fue la primera ejecución de carga real contra un contenedor `sgb_backend` que llevaba varias horas sin tráfico significativo. Esto constituye una variable extraña (*confound*) no controlada directamente por el diseño experimental – no se descartó la corrida ni se repitió hasta obtener un resultado más limpio (lo que habría sido, en sí mismo, una amenaza distinta: sesgo de selección de datos), sino que se documentó y se incluyó en el agregado de las 5 corridas, confirmando además mediante el test estadístico pareado que el patrón `caliente > frio` se sostiene igual si se considera solo el subconjunto de las corridas 2–5. Una amenaza de validez interna distinta, propia de la auditoría de seguridad ([Capítulo 2](#)), es que el mismo equipo que implementó el sistema es quien ejecutó y documentó la auditoría OWASP: no existe una segunda parte independiente que haya intentado explotar el sistema sin conocimiento previo de su código fuente, lo que puede subestimar la superficie real de vulnerabilidades frente a una prueba de penetración por un equipo externo.

5.3 Validez externa

La amenaza de validez externa más relevante y todavía sin resolver es el tamaño de muestra de la evidencia de usabilidad (System Usability Scale, SUS): la guía de esta entrega exige un mínimo de $N \geq 15$ participantes, y esa evidencia sigue **pendiente** al momento de escribir este capítulo (OBS-08 en [docs/observaciones/OBSERVACIONES.md](#), “en progreso, asignada a Panama”, sin commit real todavía). El protocolo completo – plantilla de consentimiento informado, tarea de *onboarding* de dos pasos, código de participante anónimo – ya está definido y validado, pero no ejecutado; este capítulo no puede, por tanto, evaluar la validez externa del resultado SUS en sí (todavía no existe), solo señalar el riesgo de tamaño de muestra que enfrentará cuando se ejecute. Ese riesgo no es menor: [Baltes and Ralph \(2022\)](#) documentan, en su revisión crítica de prácticas de muestreo en investigación de ingeniería de software, que muestras pequeñas y no aleatorizadas – exactamente el perfil esperable de un estudio SUS de PFC con participantes reclutados por conveniencia dentro de la comunidad universitaria – limitan severamente la generalización de cualquier conclusión más allá de la muestra concreta estudiada, y recomiendan reportar explícitamente el método de reclutamiento y sus límites en vez de presentar el resultado como representativo de una población más amplia sin justificarlo. Cuando la muestra SUS se ejecute, este capítulo deberá actualizarse citando el tamaño real obtenido, el método de reclutamiento, y aplicando ese mismo criterio de [Baltes and Ralph \(2022\)](#) a la interpretación del puntaje. Una segunda amenaza de validez externa, más estructural: todos los hallazgos de este proyecto provienen de un único contexto (una biblioteca universitaria ecuatoriana de bajo volumen, con un equipo de 3 personas sin dedicación exclusiva) – ninguna conclusión aquí debería generalizarse sin más a bibliotecas de otro tipo (públicas de gran escala, especializadas, con perfiles de carga o de usuario distintos).

5.4 Validez de conclusión

La comparación estadística pareada entre `cache_caliente` y `cache_frio` (Wilcoxon de rangos con signo sobre el p95 de cada una de las 5 corridas, `scripts/perf-analysis.py`) es el ejemplo más concreto de amenaza a la validez de conclusión de este proyecto: el estadístico exacto de dos colas con $n = 5$ pares tiene un valor mínimo alcanzable de exactamente $p = 0,0625$ cuando las 5 diferencias apuntan en la misma dirección sin excepción – que es precisamente lo que ocurre aquí (`cache_caliente > cache_frio` en las 5 corridas) –, por lo que el resultado **no alcanza el umbral convencional de significancia** ($p < 0,05$) pese a un tamaño de efecto grande y consistente (Cliff's delta = $-0,68$). Tratar este resultado como “no hay diferencia” sería una conclusión estadísticamente incorrecta: el límite real es la potencia estadística del tamaño de muestra mínimo exigido por la guía ($n = 5$ corridas), no evidencia sustantiva en contra del efecto observado – más corridas subirían la potencia sin que se espere que cambie la dirección de la conclusión, ya consistente en las 5 disponibles. Esta interpretación se documenta de forma explícita en `docs/mediciones/perf/REPORT.md` en vez de reportar solo el p -valor sin su contexto de potencia, que por sí solo induciría a la conclusión errónea de ausencia de efecto. Una segunda amenaza de conclusión, ya señalada en la sección de validez de constructo de este mismo capítulo, es que ese mismo resultado ($p = 0,0625$, efecto grande) se interpreta sobre una comparación que no aísla limpiamente el constructo de interés (el efecto del caché en sí) – ambas amenazas se refuerzan mutuamente y deben leerse juntas, no de forma aislada, para no sobre-interpretar la magnitud exacta del efecto del caché reportada aquí.

Bibliografía

Baltes, S. and Ralph, P. (2022). Sampling in software engineering research: a critical review and guidelines. *Empirical Software Engineering*, 27(4):94.